```
# 1. Import libraries
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np

from tensorflow.keras.applications import VGG16
from tensorflow.keras.applications.vgg16 import preprocess_input
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense
from tensorflow.keras.models import Model

# 2. Dataset path
dataset_path = "/home/black_hole/clg/dataset/Intel Image Classification"

# 3. Load dataset
train_data = tf.keras.utils.image_dataset_from_directory(
    dataset_path,
    image_size=(64, 64),
    batch_size=32,
    validation_split=0.2,
    subset="training",
    seed=42
)

test_data = tf.keras.utils.image_dataset_from_directory(
    dataset_path,
    image_size=(64, 64),
    batch_size=32,
    validation_split=0.2,
    subset="validation",
    seed=42
)

class_names = train_data.class_names
print("Classes:", class_names)

# 4. Preprocess images
def preprocess(images, labels):
    images = preprocess_input(images)
    return images, labels

train_data = train_data.map(preprocess)
test_data = test_data.map(preprocess)
```

```
# 5. Load pretrained VGG16
base_model = VGG16(
    weights="imagenet",
    include_top=False,
    input_shape=(64, 64, 3)
)

# Freeze base layers
base_model.trainable = False

# 6. Add custom classification layers
x = GlobalAveragePooling2D()(base_model.output)
x = Dense(128, activation="relu")(x)
output = Dense(len(class_names), activation="softmax")(x)

model = Model(base_model.input, output)

# 7. Compile model
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# 8. Train model
history = model.fit(
    train_data,
    epochs=2,
    validation_data=test_data
)

# 9. Evaluate model
loss, accuracy = model.evaluate(test_data)
print("Test Accuracy:", accuracy)

# 10. Fine-tune model
base_model.trainable = True

for layer in base_model.layers:
    if not layer.name.startswith("block5"):
        layer.trainable = False

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.00001),
```

```
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.fit(
    train_data,
    epochs=1,
    validation_data=test_data
)

# 11. Final evaluation
loss, accuracy = model.evaluate(test_data)
print("Final Accuracy:", accuracy)

# 12. Plot accuracy
plt.plot(history.history["accuracy"], label="Training")
plt.plot(history.history["val_accuracy"], label="Validation")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

# 13. Display predictions
for images, labels in test_data.take(1):

    predictions = model.predict(images)

    plt.figure(figsize=(10, 6))

    for i in range(6):
        img = images[i].numpy()
        img = img + [103.939, 116.779, 123.68]
        img = img[..., ::-1]
        img = np.clip(img, 0, 255).astype("uint8")

        plt.subplot(2, 3, i + 1)
        plt.imshow(img)

        actual = class_names[labels[i].numpy()]
        predicted = class_names[np.argmax(predictions[i])]

        plt.title("Actual: " + actual + "\nPredicted: " + predicted)
        plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```